Práctica 2

Sistemas Operativos

José Miguel Martínez García

9 de Noviembre de 2026

Github: https://github.com/Schwarovsky/SO/tree/main

Índice

1. Enunciado del Ejercicio 4
1.1. Estructura del Árbol de Procesos 5
1.2. Resultado Esperado 5
2. Justificación e Identificación de tareas 6
2.1. Tareas realizadas por el servidor 6
2.1.1. Creación y configuración del socket 6
2.1.2. Asociación del socket y puesta en escucha 7
2.1.3. Aceptación de conexiones y creación de procesos hijos 7
2.1.4. Lectura y transmisión del fichero 8
2.1.5. Cierre de conexiones y finalización de procesos 8
2.2. Tareas realizadas por el cliente 9
2.2.1. Recepción y validación de argumentos 9
2.2.2. Creación del socket 9
2.2.3. Configuración de la dirección y establecimiento de la conexión 9
2.2.4. Recepción y visualización de los datos 10
2.2.5. Cierre del socket 10
3. Diagrama de Secuencia 10
4. Análisis detallado del código fuente 11
4.1. Análisis del programa Servidor.c 11
4.1.1. Función crearServidor() 11
4.1.2. Función transmitirFichero() 14
4.1.3. Función atenderCliente() 17
4.1.4. Función esperarClientes() 19
4.1.5. Función main() 22
4.2. Análisis del programa Cliente.c 24
4.2.1. Función comprobarArgumentos() 24
4.2.2. Función crearSocket() 25
4.2.3. Función conectarServidor() 26
4.2.4. Función recibirDatos() 28
4.2.5. Función main() 30
1. Enunciado del ejercicio 32
1.1. Estructura del Árbol de Procesos 32
1.2. Resultado Esperado 32
2. Justificación e Identificación de Tareas 35
2.1. Tareas realizadas por la máquina productora 36
2.1.1. Inicio del proceso de producción 36

2.1.2. Comprobación de los espacios disponibles 36
2.1.3. Acceso exclusivo a la cinta transportadora 37
2.1.4. Creación y depósito de los paquetes 37
2.1.5. Finalización de la producción 38
2.2. Tareas realizadas por los robots consumidores 38
2.2.1. Inicio del proceso de consumo 38
2.2.2. Comprobación de las piezas disponibles 39
2.2.3. Acceso exclusivo a la cinta transportadora 39
2.2.4. Recogida y traslado de los paquetes 40
2.2.5. Liberación de recursos y notificación de espacios disponibles 40
2.2.6. Finalización del consumo 41
2.3. Mecanismos de sincronización empleados 41
2.3.1. Semáforo huecos 41
2.3.2. Semáforo paquetes 41
2.3.3. Semáforo mutex 42
3. Diagrama de Secuencia 42
4. Análisis detallado del Código fuente 43
4.1. Declaración de constantes y variables globales 43
4.2. Función maquina() 45
4.3. Función robot() 49
4.4. Función main() 53

1. Ejercicio 1

Comunicaciones en Red

1. Enunciado del Ejercicio

El ejercicio constará de dos partes (procesos):

- El servidor. Proceso que estará esperando a que le lleguen conexiones desde el

  cliente y, una vez establecida la conexión, transferirá al cliente el contenido de

  un archivo denominado “Google.html”. El contenido del archivo será el código

  fuente de la página web de Google.
- El cliente. Proceso que establecerá la conexión con el servidor, recibirá el

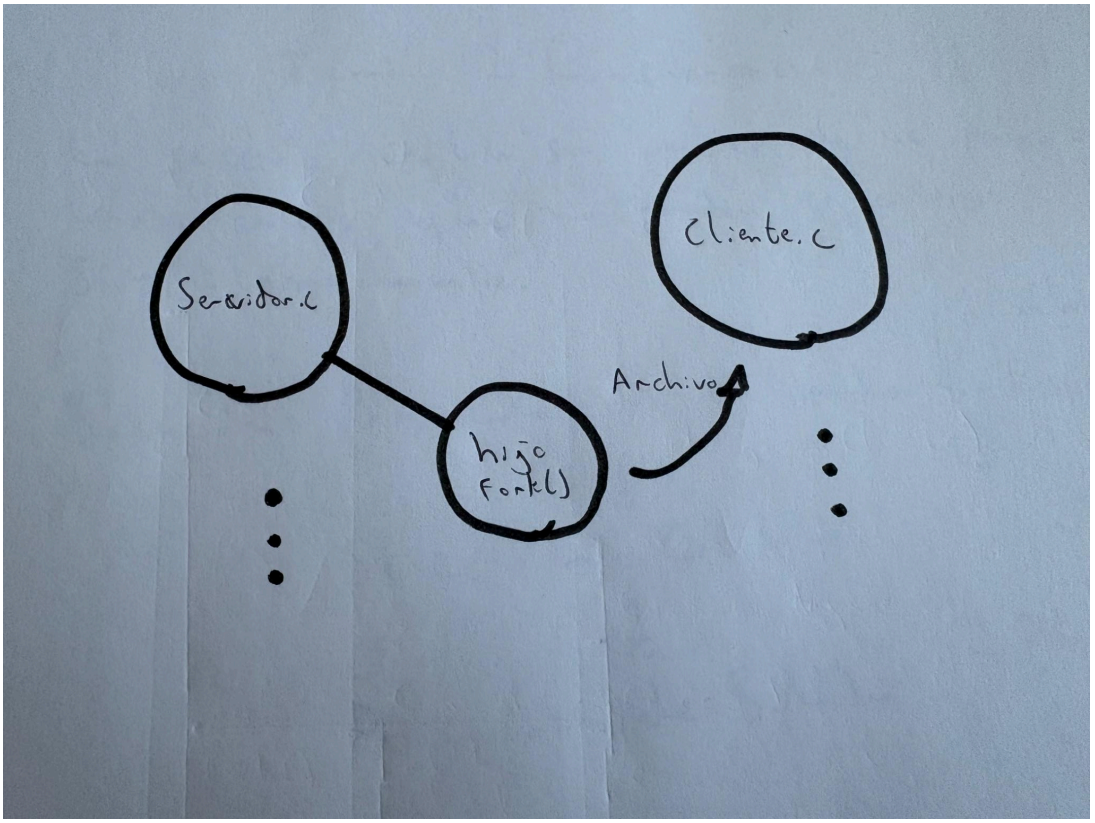
  contenido del archivo y lo mostrará por pantalla.

El cliente se ejecutará en la máquina local y se invocará con la orden:

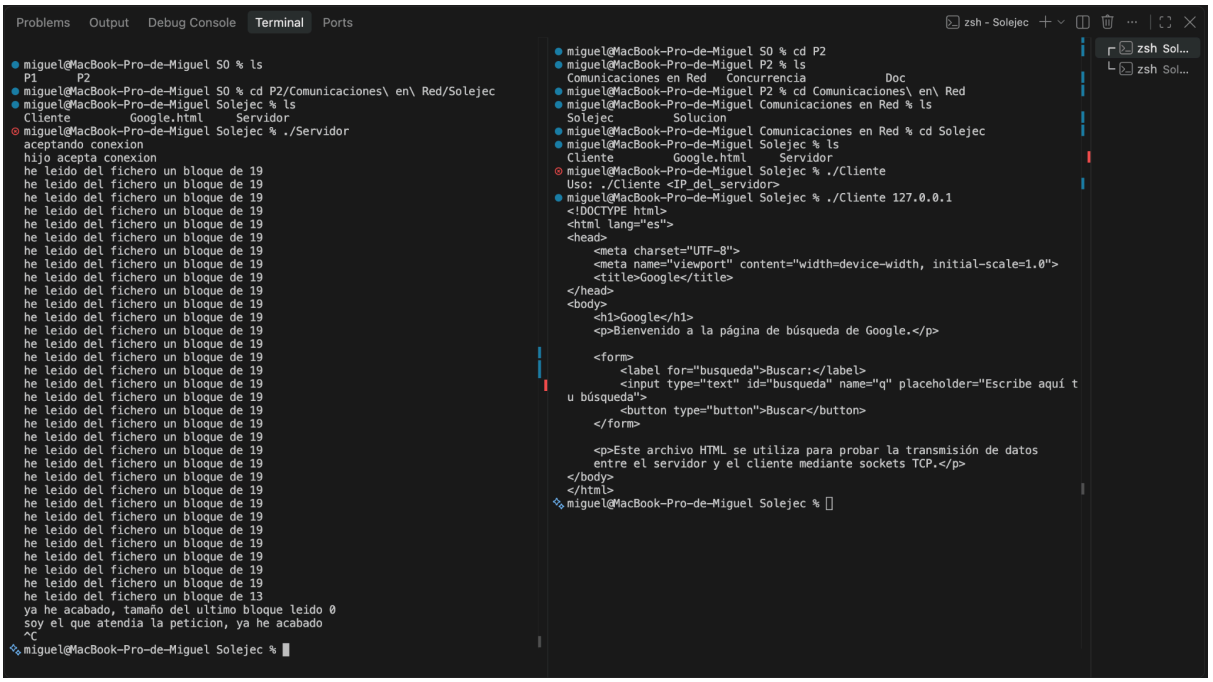
Cliente IP_Servidor

El servidor se lanzará con la orden **Servidor** y debe estar en todo momento escuchando por el puerto 9999, preparado para aceptar conexiones. Tras una petición de conexión por parte de un cliente, debe crear un hijo que será el encargado de realizar todas las operaciones necesarias para la transferencia del archivo.

1.1. Estructura del Árbol de Procesos



1.2. Resultado Esperado



2. Justificación e Identificación de tareas

El programa desarrollado consiste en una aplicación cliente-servidor que permite transmitir el contenido de un fichero HTML desde un servidor hacia uno o varios clientes mediante el uso de sockets TCP en lenguaje C.

El servidor se encarga de establecer un punto de comunicación a través de un puerto determinado, permanecer a la espera de conexiones entrantes y atender las peticiones recibidas. Para permitir la atención simultánea de varios clientes, se emplea la creación de procesos hijos mediante la llamada al sistema fork(), de manera que cada proceso hijo se ocupa de transmitir el contenido del fichero

solicitado sin interrumpir la actividad del proceso padre.

Por su parte, el cliente recibe como argumento la dirección IP del servidor, establece una conexión con él y recibe los datos transmitidos, mostrándolos posteriormente por pantalla.

A continuación, se identifican las principales tareas realizadas por cada programa.

2.1. Tareas realizadas por el servidor

2.1.1. Creación y configuración del socket

En primer lugar, el servidor crea un socket de tipo TCP mediante el cual podrá establecer comunicaciones con los clientes. Posteriormente, configura su dirección de red, establece el puerto de comunicación 9999 y habilita la reutilización de la dirección mediante las opciones del socket.

2.1.2. Asociación del socket y puesta en escucha

Una vez configurado, el servidor vincula el socket a la dirección y al puerto establecidos. A continuación, lo pone en estado de escucha para poder recibir solicitudes de conexión de los clientes, manteniendo una cola de conexiones pendientes con una capacidad indicada de 5.

2.1.3. Aceptación de conexiones y creación de procesos hijos

El servidor permanece ejecutándose en un bucle continuo, esperando nuevas conexiones mediante la función accept(). Cuando recibe una petición, crea un proceso hijo utilizando fork().

El proceso padre continúa esperando nuevas conexiones, mientras que el proceso hijo se encarga de atender al cliente que acaba de conectarse. De esta manera, el servidor puede gestionar distintas conexiones sin tener que esperar a que finalice la transmisión anterior.

2.1.4. Lectura y transmisión del fichero

Cada proceso hijo abre el fichero Google.html en modo de solo lectura y transmite su contenido al cliente correspondiente.

La lectura se realiza en bloques de un máximo de 19 bytes, que posteriormente se envían mediante el socket de comunicación. Se contempla la posibilidad de que una operación de escritura no transmita todos los bytes solicitados, por lo que se repite hasta completar el envío del bloque o detectar un error.

2.1.5. Cierre de conexiones y finalización de procesos

Cuando termina la transmisión, el proceso hijo cierra el fichero y el socket utilizado para comunicarse con el cliente, finalizando posteriormente su ejecución.

Mientras tanto, el proceso padre continúa aceptando nuevas conexiones. Además, se ignora la señal SIGCHLD para evitar la acumulación de procesos hijos zombis y la señal SIGPIPE para impedir que una desconexión inesperada del cliente termine el proceso al intentar transmitir datos.

2.2. Tareas realizadas por el cliente

2.2.1. Recepción y validación de argumentos

El cliente recibe mediante la línea de comandos la dirección IP del servidor al que desea conectarse. Antes de comenzar la comunicación, comprueba que se haya proporcionado exactamente un argumento adicional, correspondiente a dicha dirección.

2.2.2. Creación del socket

Una vez validado el argumento, el cliente crea un socket de tipo TCP que utilizará para establecer la comunicación con el servidor.

2.2.3. Configuración de la dirección y establecimiento de la conexión

El cliente configura la estructura que contiene la dirección del servidor, indicando el protocolo IPv4, el puerto de comunicación 9999 y la dirección IP recibida como argumento.

Posteriormente, intenta establecer la conexión con el servidor mediante la función connect(). Si la operación falla, se muestra el correspondiente mensaje de error y se cierra el socket.

2.2.4. Recepción y visualización de los datos

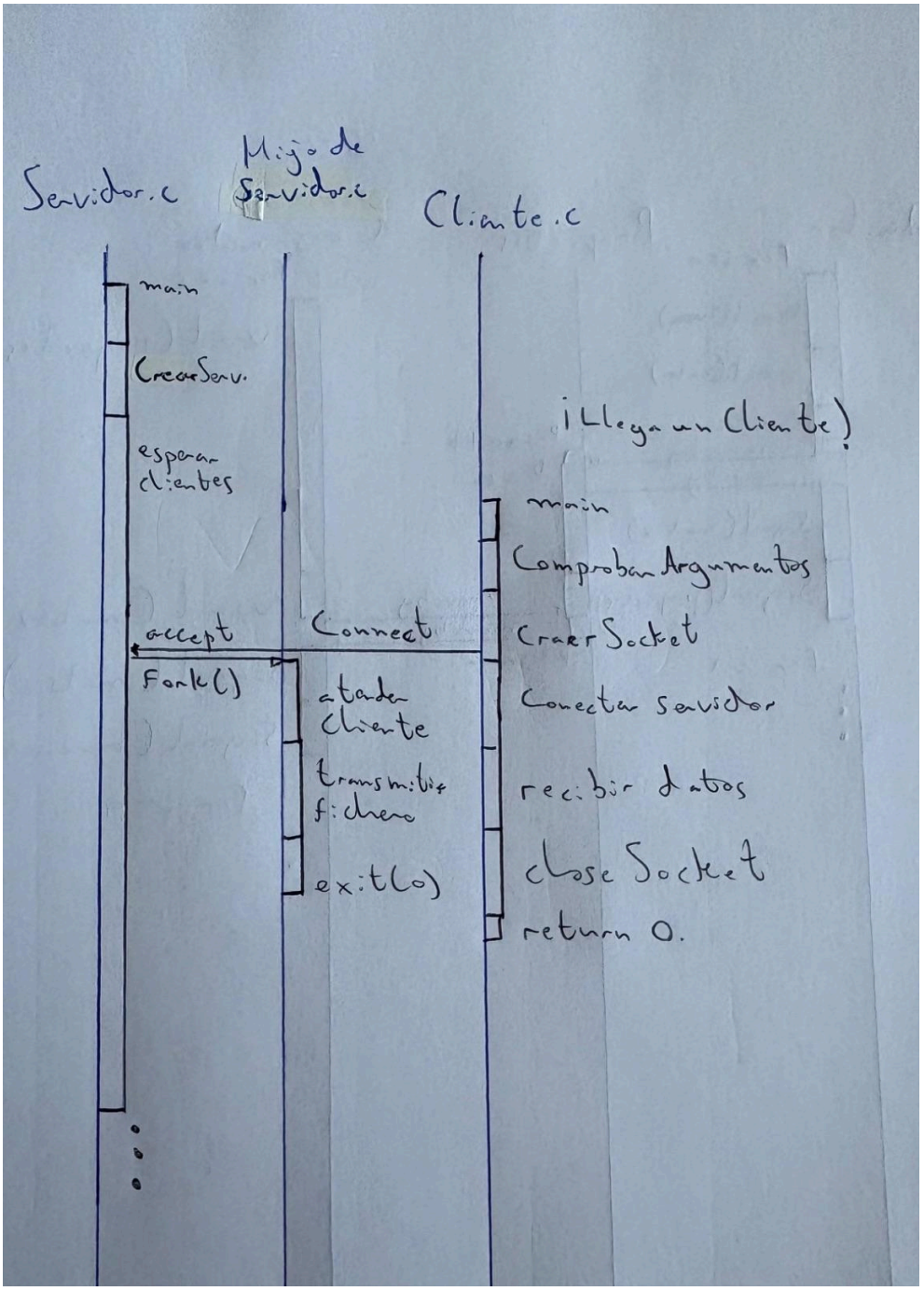
Una vez establecida la conexión, el cliente comienza a recibir los datos enviados por el servidor mediante operaciones sucesivas de lectura sobre el socket.

Los datos se reciben en bloques de un máximo de 19 bytes y se muestran por pantalla a medida que llegan. Este procedimiento continúa hasta que el servidor termina la transmisión y se alcanza el final de la comunicación.

2.2.5. Cierre del socket

Finalmente, cuando termina la recepción de los datos, el cliente cierra el socket utilizado para la comunicación y finaliza su ejecución.

3. Diagrama de Secuencia



4. Análisis detallado del código fuente

4.1. Análisis del programa Servidor.c

4.1.1. Función crearServidor()

```
22  int crearServidor(){
23
24      struct sockaddr_in serverAddress;
25      int sock;
26      int opcion = 1;
27
28      sock = socket(AF_INET, SOCK_STREAM, 0);
29
30      if(sock < 0){
31          perror("Error. No se pudo abrir el socket");
32          return -1;
33      }
34
35      bzero((char *) &serverAddress, sizeof(serverAddress));
36
37      serverAddress.sin_port = htons(ConstPuerto);
38      serverAddress.sin_family = AF_INET;
39      serverAddress.sin_addr.s_addr = INADDR_ANY;
40
41      setsockopt(sock, SOL_SOCKET, SO_REUSEADDR, &opcion, sizeof(opcion));
42
43      if(bind(sock, (struct sockaddr *) &serverAddress, sizeof(serverAddress)) < 0){
44          perror("Error no se pudo vincular el socket");
45          close(sock);
46          return -1;
47      }
48
49      if(listen(sock, 5) < 0){
50          perror("error en listen");
51          close(sock);
52          return -1;
53      }
54
55      return sock;
56  }
```

Esta función se encarga de crear y configurar el socket que utilizará el servidor para recibir conexiones entrantes. No recibe ningún parámetro y devuelve un descriptor de fichero de tipo entero correspondiente al socket creado. En caso de

producirse algún error durante la creación, vinculación o puesta en escucha, devuelve -1.

En primer lugar, declara una estructura sockaddr_in denominada serverAddress, que almacenará la información de la dirección del servidor. También declara la variable sock, que contendrá el descriptor del socket, y opcion, inicializada a 1, que se utiliza para configurar la reutilización de la dirección.

A continuación, se invoca la función socket(AF_INET, SOCK_STREAM, 0), que crea un socket de comunicación utilizando IPv4 (AF_INET) y el protocolo TCP (SOCK_STREAM). Si la creación falla, se muestra un mensaje de error mediante perror() y se devuelve -1.

Posteriormente, se inicializa a cero la estructura serverAddress mediante bzero(). Después se configuran sus campos:

- sin_port: establece el puerto 9999 mediante htons(), que convierte el número al orden de bytes de red.
- sin_family: indica que se utilizará IPv4 mediante AF_INET.
- sin_addr.s_addr: establece INADDR_ANY, permitiendo que el servidor acepte conexiones dirigidas a cualquiera de las interfaces de red disponibles.

Una vez configurada la dirección, se utiliza setsockopt() con las opciones SOL_SOCKET y

SO_REUSEADDR para permitir la reutilización de la dirección local del socket.

A continuación, se ejecuta bind(), que asocia el socket creado con la dirección y el puerto configurados. Si esta operación falla, se muestra un mensaje de error, se cierra el socket mediante close() y se devuelve -1.

Finalmente, se llama a listen() para poner el socket en estado de escucha, estableciendo una capacidad de 5 conexiones pendientes. Si se produce algún error, se cierra el socket y se devuelve -1. En caso contrario, la función devuelve el descriptor sock, que será utilizado posteriormente por el servidor para aceptar conexiones.

4.1.2. Función transmitirFichero()

```
61  void transmitirFichero(int newSock){
62
63      ssize_t leidos, enviados, escritos;
64      int fichOrigen;
65      char buffer[ConstTam];
66
67      fichOrigen = open("Google.html", O_RDONLY);
68
69      if(fichOrigen < 0){
70          perror("error al abrir el fichero");
71      }
72      else{
73
74          // TRANSMISION DEL FICHERO.
75
76          leidos = read(fichOrigen, buffer, ConstTam - 1);
77
78          while(leidos > 0){
79
80              printf("he leido del fichero un bloque de %zd\n", leidos);
81
82              enviados = 0;
83
84              while(enviados < leidos){
85
86                  escritos = write(newSock, buffer + enviados, leidos - enviados);
87
88                  if(escritos <= 0){
89                      perror("error al enviar al cliente");
90                      break;
91                  }
92
93                  enviados += escritos;
94              }
95
96              if(enviados < leidos) break;
97
98              leidos = read(fichOrigen, buffer, ConstTam - 1);
99          }
100
101         if(leidos < 0) perror("error al leer el fichero");
102
103         printf("ya he acabado, tamaño del ultimo bloque leido %zd\n", leidos);
104
105         close(fichOrigen);
106     }
107 }
```

Esta función se encarga de leer el contenido del fichero Google.html y transmitirlo al cliente mediante el socket de comunicación. Recibe como parámetro

newSock, que representa el descriptor del socket devuelto por accept() y utilizado para comunicarse con el cliente.

En primer lugar, declara tres variables de tipo ssize_t: leidos, enviados y escritos. Estas variables permiten controlar, respectivamente, la cantidad de bytes leídos del fichero, los bytes que ya se han enviado y los bytes transmitidos en cada operación de escritura.

También declara fichOrigen, que almacenará el descriptor del fichero abierto, y buffer, un array de caracteres de tamaño ConstTam, cuyo valor es 20.

A continuación, se utiliza open("Google.html", O_RDONLY) para abrir el fichero en modo de solo lectura. Si la apertura falla, se muestra un mensaje de error mediante perror(). En caso contrario, comienza el procedimiento de transmisión.

Primero se realiza una lectura mediante read(), solicitando un máximo de ConstTam - 1 bytes, es decir, 19 bytes. El resultado se almacena en leidos.

Mientras leidos sea mayor que cero, significa que se han obtenido datos del fichero y que deben transmitirse al cliente. En cada iteración se muestra por pantalla el número de bytes leídos y se inicializa enviados a cero.

Posteriormente, se ejecuta un segundo bucle encargado de transmitir completamente el bloque leído. Para ello, se utiliza write(newSock, buffer +

enviados, leidos - enviados), que envía los bytes pendientes a través del socket.

El uso de buffer + enviados permite continuar la escritura desde la posición del array en la que quedó la transmisión anterior, mientras que leidos - enviados indica el número de bytes que todavía quedan por enviar.

Si write() devuelve un valor menor o igual que cero, se muestra un mensaje de error y se interrumpe el bucle de escritura. En caso contrario, el número de bytes transmitidos se acumula en enviados.

Una vez finalizado el bucle interno, se comprueba si se han enviado todos los bytes del bloque. Si quedan datos pendientes, se interrumpe la transmisión. Si el bloque se ha enviado completamente, se realiza una nueva lectura del fichero para obtener el siguiente bloque.

Este procedimiento continúa hasta que read() devuelve cero, indicando que se ha alcanzado el final del fichero. Si devuelve un valor negativo, se muestra un mensaje de error de lectura.

Finalmente, se muestra por pantalla el tamaño devuelto por la última lectura y se cierra el fichero mediante close(fichOrigen).

La función no devuelve ningún valor, ya que su finalidad es realizar directamente la transmisión de los datos.

4.1.3. Función atenderCliente()

```
110  // Atiende la petición de un cliente desde el proceso hijo.
111
112  void atenderCliente(int sock, int newSock){
113
114      close(sock); // El hijo no acepta nuevas conexiones.
115
116      printf("hijo acepta conexion\n");
117
118      transmitirFichero(newSock);
119
120      printf("soy el que atendia la peticion, ya he acabado\n");
121
122      close(newSock);
123
124      fflush(stdout);
125
126      exit(0); // El hijo no vuelve a accept.
127  }
128
```

Esta función se ejecuta exclusivamente en el proceso hijo creado mediante fork() y se encarga de atender la petición del cliente que ha establecido la conexión.

Recibe dos parámetros:

- sock: descriptor del socket principal del servidor, utilizado para aceptar nuevas conexiones.
- newSock: descriptor del nuevo socket obtenido mediante accept(), utilizado para comunicarse con el cliente.

En primer lugar, ejecuta close(sock) para cerrar la copia del socket principal que posee el proceso hijo. De esta forma, el hijo no mantiene abierto el socket

destinado a aceptar nuevas conexiones, ya que su responsabilidad consiste únicamente en atender al cliente asignado.

A continuación, muestra un mensaje indicando que el proceso hijo ha aceptado la conexión y llama a transmitirFichero(newSock) para enviar el contenido de Google.html al cliente.

Cuando termina la transmisión, muestra un mensaje indicando que ha finalizado la atención de la petición y cierra el socket de comunicación mediante close(newSock).

Posteriormente, ejecuta fflush(stdout) para vaciar el búfer de salida estándar y garantizar que los mensajes pendientes se escriban antes de terminar el proceso.

Finalmente, utiliza exit(0) para finalizar la ejecución del proceso hijo, evitando que este continúe ejecutando el bucle de aceptación de conexiones del servidor.

La función no devuelve ningún valor, ya que su objetivo es gestionar completamente la atención de una conexión desde el proceso hijo.

4.1.4. Función esperarClientes()

```
132  void esperarClientes(int sock){
133
134      socklen_t tam;
135      struct sockaddr_in clientAddress;
136      int newSock;
137      pid_t pid;
138
139      signal(SIGCHLD, SIG_IGN); // Evita hijos zombis.
140      signal(SIGPIPE, SIG_IGN); // Evita que un cliente desconectado termine el proceso.
141
142      do{
143
144          tam = sizeof(clientAddress);
145
146          newSock = accept(sock, (struct sockaddr *) &clientAddress, &tam);
147
148          if(newSock < 0){
149
150              perror("error al aceptar la conexion");
151          }
152          else{
153
154              printf("aceptando conexion\n");
155              fflush(stdout);
156
157              pid = fork();
158
159              if(pid == 0){
160
161                  atenderCliente(sock, newSock);
162              }
163              else if(pid > 0){
164
165                  close(newSock); // El padre no transmite el fichero.
166              }
167              else{
168
169                  perror("error en fork");
170                  close(newSock);
171              }
172          }
173
174      }while(1);
175  }
```

Esta función se encarga de mantener el servidor a la espera de nuevas conexiones y crear un proceso hijo para atender cada cliente recibido.

Recibe como parámetro sock, que representa el descriptor del socket principal del servidor, previamente creado y configurado por crearServidor().

En primer lugar, declara las variables necesarias para gestionar las conexiones:

- tam: variable de tipo socklen_t que almacena el tamaño de la estructura correspondiente a la dirección del cliente.
- clientAddress: estructura sockaddr_in que almacena la información de la dirección del cliente.
- newSock: descriptor del nuevo socket generado al aceptar una conexión.
- pid: variable de tipo pid_t que almacena el resultado de la llamada a fork().

A continuación, se utilizan dos llamadas a signal() para modificar el comportamiento del servidor ante determinadas señales.

La primera, signal(SIGCHLD, SIG_IGN), establece que se ignore la señal SIGCHLD, generada cuando finaliza un proceso hijo. Esto permite evitar la acumulación de procesos hijos zombis.

La segunda, signal(SIGPIPE, SIG_IGN), establece que se ignore la señal SIGPIPE, que puede producirse cuando se intenta escribir en una conexión cuyo extremo receptor se ha cerrado. De esta forma, se evita que el proceso termine automáticamente por recibir dicha señal.

Posteriormente, comienza un bucle infinito mediante do...while(1), que permite al servidor permanecer continuamente a la espera de nuevas conexiones.

En cada iteración, se inicializa tam con el tamaño de clientAddress y se ejecuta accept(). Esta función bloquea la ejecución hasta que llega una petición de conexión y, cuando la acepta, devuelve un nuevo descriptor de socket que se almacena en newSock.

Si accept() devuelve un valor negativo, se muestra un mensaje de error mediante perror() y el servidor continúa ejecutando el bucle.

Si la conexión se acepta correctamente, se muestra un mensaje por pantalla y se utiliza fflush(stdout) para vaciar el búfer de salida estándar.

A continuación, se ejecuta fork() para crear un nuevo proceso. Dependiendo del valor devuelto por esta llamada, se distinguen tres situaciones:

- Si pid == 0: se está ejecutando el proceso hijo. Este llama a atenderCliente(sock, newSock), encargándose de transmitir el fichero al cliente y finalizar posteriormente su ejecución.
- Si pid > 0: se está ejecutando el proceso padre. Este cierra su copia del socket newSock, ya que no necesita utilizarlo para transmitir datos. De esta forma, puede continuar esperando nuevas conexiones mediante accept().
- Si pid < 0: se ha producido un error al crear el proceso hijo. Se muestra un mensaje mediante

perror() y se cierra el socket de la conexión que no ha podido atenderse.

La función no devuelve ningún valor, puesto que su finalidad es mantener el servidor funcionando continuamente y gestionar las conexiones entrantes.

4.1.5. Función main()

```
178  int main(){
179
180      int sock;
181
182      sock = crearServidor();
183
184      if(sock < 0){
185
186          return 1;
187      }
188
189      esperarClientes(sock);
190
191      return 0;
192  }
193
```

Es la función principal del programa servidor y se encarga de iniciar y coordinar su funcionamiento.

En primer lugar, declara la variable sock, que almacenará el descriptor del socket principal del servidor.

A continuación, llama a crearServidor() para crear y configurar el socket. El resultado de esta operación se almacena en sock.

Posteriormente, comprueba si el valor devuelto es menor que cero. En caso afirmativo, significa que se ha producido un error durante la inicialización del servidor, por lo que el programa finaliza devolviendo el valor 1.

Si la creación del socket se ha realizado correctamente, se llama a esperarClientes(sock), que mantiene el servidor en funcionamiento y se encarga de aceptar conexiones y crear procesos hijos para atenderlas.

Finalmente, se incluye return 0 como instrucción de finalización normal del programa, aunque en condiciones habituales no se alcanza debido al bucle infinito de esperarClientes().

4.2. Análisis del programa Cliente.c

4.2.1. Función comprobarArgumentos()

```
22  bool comprobarArgumentos(int argc, char *argv[]){
23
24      if(argc != 2){
25          fprintf(stderr, "Uso: %s <IP_del_servidor>\n", argv[0]);
26          return false;
27      }
28
29      return true;
30  }
```

Esta función se encarga de comprobar que el usuario ha introducido correctamente los argumentos necesarios para ejecutar el cliente.

Recibe dos parámetros:

- argc: número de argumentos introducidos en la línea de comandos, incluyendo el nombre del propio programa.
- argv: array de cadenas de caracteres que contiene los argumentos recibidos.

En primer lugar, comprueba si argc es distinto de 2. Esto significa que el usuario no ha proporcionado exactamente un argumento adicional, correspondiente a la dirección IP del servidor.

Si la condición se cumple, se muestra un mensaje de uso mediante fprintf(stderr, ...), indicando el formato correcto de ejecución. Posteriormente, devuelve false para señalar que los argumentos no son válidos.

En caso contrario, devuelve true, indicando que se ha proporcionado el número de argumentos esperado.

La función utiliza el tipo bool, definido mediante la biblioteca stdbool.h, y devuelve un valor booleano que permite controlar si el programa puede continuar su ejecución.

4.2.2. Función crearSocket()

```
35  int crearSocket(void){
36
37      int sock;
38
39      sock = socket(AF_INET, SOCK_STREAM, 0);
40
41      if(sock < 0){
42          perror("Error. No se pudo crear socket");
43      }
44
45      return sock;
46  }
```

Esta función se encarga de crear el socket que utilizará el cliente para comunicarse con el servidor.

No recibe ningún parámetro y declara una variable entera denominada sock, donde se almacenará el descriptor del socket.

A continuación, ejecuta socket(AF_INET, SOCK_STREAM, 0), creando un socket IPv4 de tipo TCP.

Si el resultado es menor que cero, significa que se ha producido un error durante la creación del socket, por lo que se muestra el correspondiente mensaje mediante perror().

Finalmente, devuelve el valor de sock. Si la creación se ha realizado correctamente, devuelve el descriptor del socket; en caso contrario, devuelve -1.

4.2.3. Función conectarServidor()

```
51  bool conectarServidor(int sock, char *ip){
52
53      struct sockaddr_in serv_addr;
54
55      bzero((char *) &serv_addr, sizeof(serv_addr));
56
57      serv_addr.sin_port = htons(ConstPuerto);
58      serv_addr.sin_family = AF_INET;
59      serv_addr.sin_addr.s_addr = inet_addr(ip);
60
61      if(connect(sock, (struct sockaddr *) &serv_addr, sizeof(serv_addr)) < 0){
62          perror("Error. No se pudo conectar al socket");
63          close(sock);
64          return false;
65      }
66
67      return true;
68  }
```

Esta función se encarga de configurar la dirección del servidor y establecer la conexión TCP con él.

Recibe dos parámetros:

- sock: descriptor del socket previamente creado mediante crearSocket().
- ip: cadena de caracteres que contiene la dirección IP del servidor, recibida como argumento del programa.

En primer lugar, declara una estructura sockaddr_in denominada serv_addr, que almacenará la información necesaria para establecer la conexión.

A continuación, inicializa la estructura a cero mediante bzero() y configura sus campos:

- sin_port: establece el puerto 9999 mediante htons().
- sin_family: indica que se utilizará el protocolo IPv4 mediante AF_INET.
- sin_addr.s_addr: convierte la dirección IP recibida como argumento a su representación binaria mediante inet_addr().

Una vez configurada la dirección, se ejecuta connect(), que intenta establecer una conexión con el servidor utilizando el socket creado anteriormente.

Si la operación devuelve un valor menor que cero, significa que no ha sido posible establecer la conexión. En ese caso, se muestra un mensaje de error mediante perror(), se cierra el socket utilizando close(sock) y se devuelve false.

Si la conexión se establece correctamente, la función devuelve true, indicando que el cliente puede comenzar a recibir los datos enviados por el servidor.

4.2.4. Función recibirDatos()

```
73  void recibirDatos(int sock){
74
75      char buffer[ConstTam];
76      ssize_t tam;
77
78      do{
79
80          tam = read(sock, buffer, ConstTam - 1);
81
82          if(tam > 0){
83
84              buffer[tam] = '\0';
85
86              printf("%s", buffer);
87          }
88
89      }while(tam > 0);
90
91      if(tam < 0){
92          perror("error al leer del socket");
93      }
94
95  }
```

Esta función se encarga de recibir el contenido transmitido por el servidor y mostrarlo por pantalla.

Recibe como parámetro sock, que representa el descriptor del socket mediante el cual se ha establecido la conexión con el servidor.

En primer lugar, declara un array de caracteres denominado buffer, con un tamaño de ConstTam,

cuyo valor es 20. También declara la variable tam, de tipo ssize_t, que almacenará el número de bytes recibidos en cada operación de lectura.

A continuación, comienza un bucle do...while en el que se ejecuta la función read(sock, buffer, ConstTam - 1). Esta operación intenta recibir un máximo de 19 bytes del servidor y almacena el resultado en tam.

Si tam es mayor que cero, significa que se han recibido datos correctamente. En ese caso, se añade el carácter terminador '\0' en la posición correspondiente del buffer, convirtiendo los datos recibidos en una cadena de caracteres terminada en nulo.

Posteriormente, se utiliza printf("%s", buffer) para mostrar por pantalla el contenido recibido.

El bucle continúa mientras tam sea mayor que cero, es decir, mientras sigan llegando datos del servidor. Cuando read() devuelve cero, significa que se ha alcanzado el final de la transmisión y el servidor ha cerrado su extremo de la conexión.

Si read() devuelve un valor negativo, se muestra un mensaje de error mediante perror().

La función no devuelve ningún valor, ya que su finalidad es recibir y mostrar directamente los datos transmitidos.

4.2.5. Función main()

```
100  int main(int argc, char *argv[]){
101
102      int sock;
103
104      if(!comprobarArgumentos(argc, argv)){
105          return 1;
106      }
107
108      sock = crearSocket();
109
110      if(sock < 0){
111          return 1;
112      }
113
114      if(conectarServidor(sock, argv[1])){
115
116          recibirDatos(sock);
117
118          close(sock);
119      }
120
121      return 0;
122  }
123
```

Es la función principal del cliente y se encarga de coordinar las distintas operaciones necesarias para establecer la comunicación y recibir el fichero enviado por el servidor.

Recibe los parámetros argc y argv, que contienen el número de argumentos introducidos por el usuario y sus respectivos valores.

En primer lugar, declara la variable sock, que almacenará el descriptor del socket del cliente.

A continuación, llama a comprobarArgumentos(argc, argv) para verificar que se ha proporcionado la dirección IP del servidor. Si la función devuelve false, el programa finaliza devolviendo 1, ya que no dispone de los argumentos necesarios para continuar.

Si los argumentos son correctos, se llama a crearSocket() para crear el socket de comunicación. El descriptor devuelto se almacena en sock. Si su valor es menor que cero, significa que se ha producido un error y el programa finaliza devolviendo 1.

Posteriormente, se llama a conectarServidor(sock, argv[1]), pasando como parámetros el descriptor del socket y la dirección IP proporcionada por el usuario.

Si la conexión se establece correctamente, se llama a recibirDatos(sock), que se encarga de recibir y mostrar por pantalla el contenido transmitido por el servidor.

Una vez finalizada la recepción, se cierra el socket mediante close(sock).

Finalmente, el programa devuelve 0, indicando que ha terminado su ejecución.

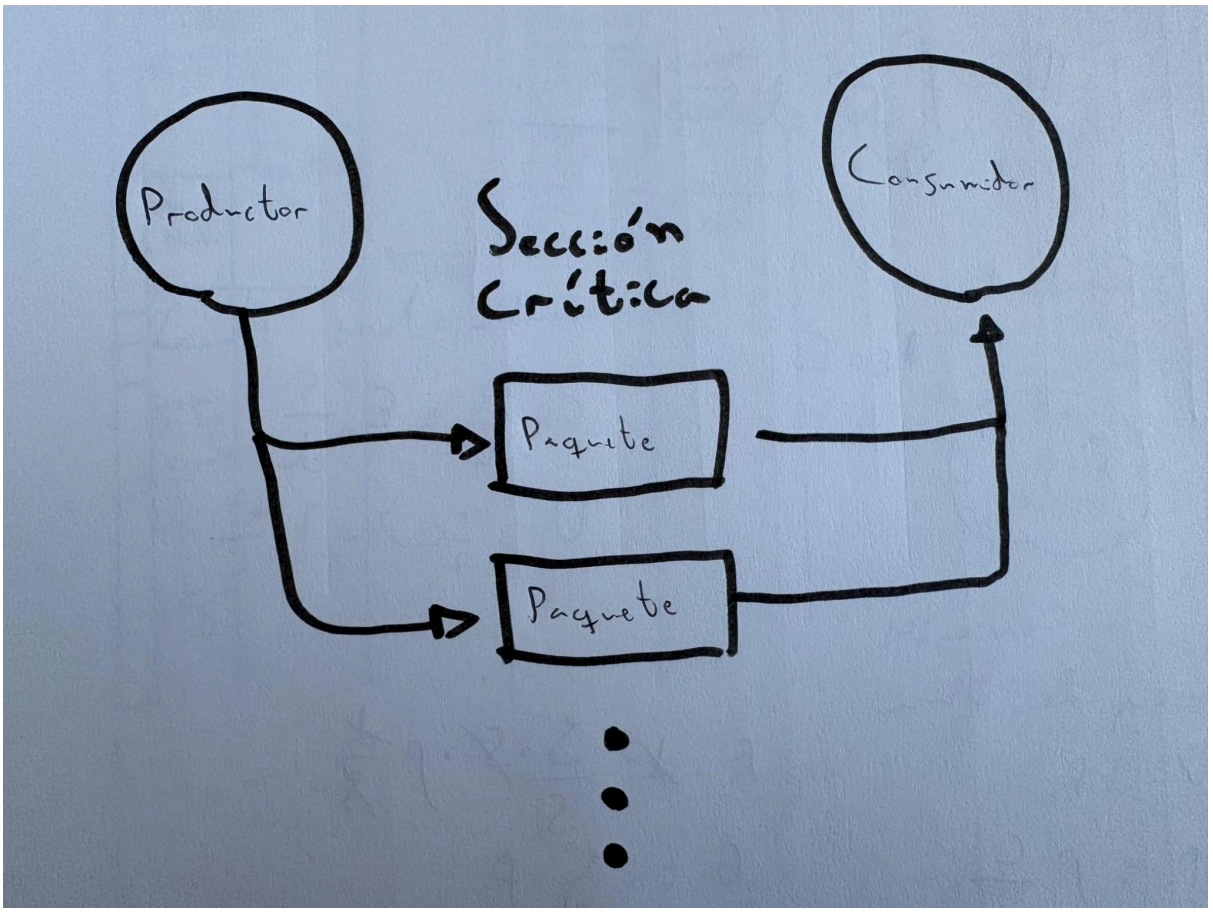
2. Ejercicio 2

Concurrencia

1. Enunciado del ejercicio

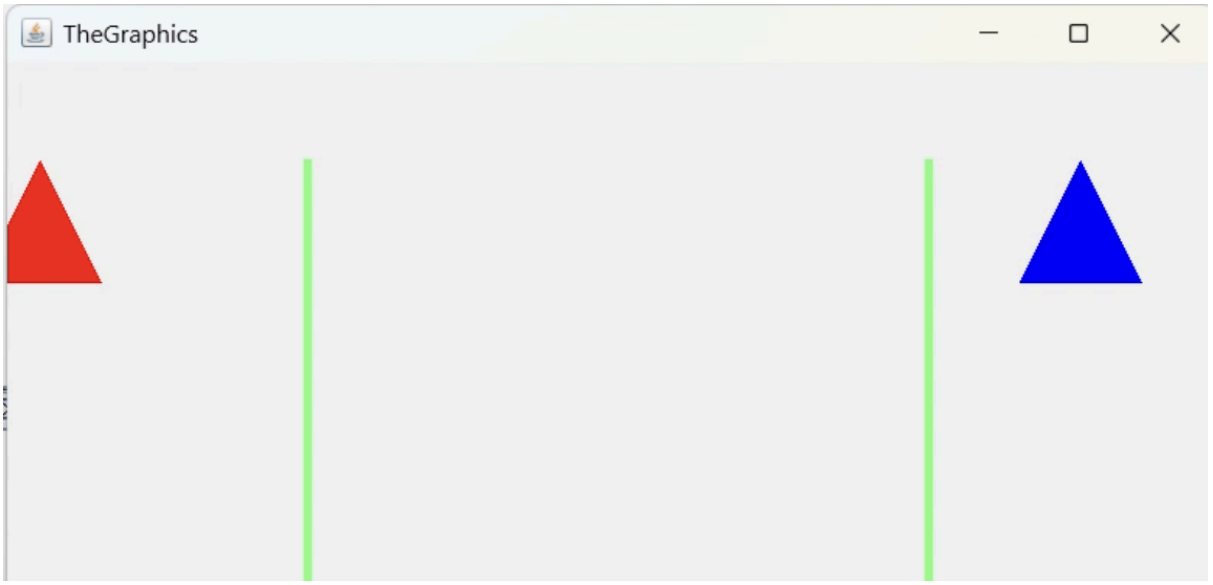
Implementar mediante semáforos el problema del productor consumidor con buffer limitado estudiado en clase de teoría.

1.1. Estructura del Árbol de Procesos



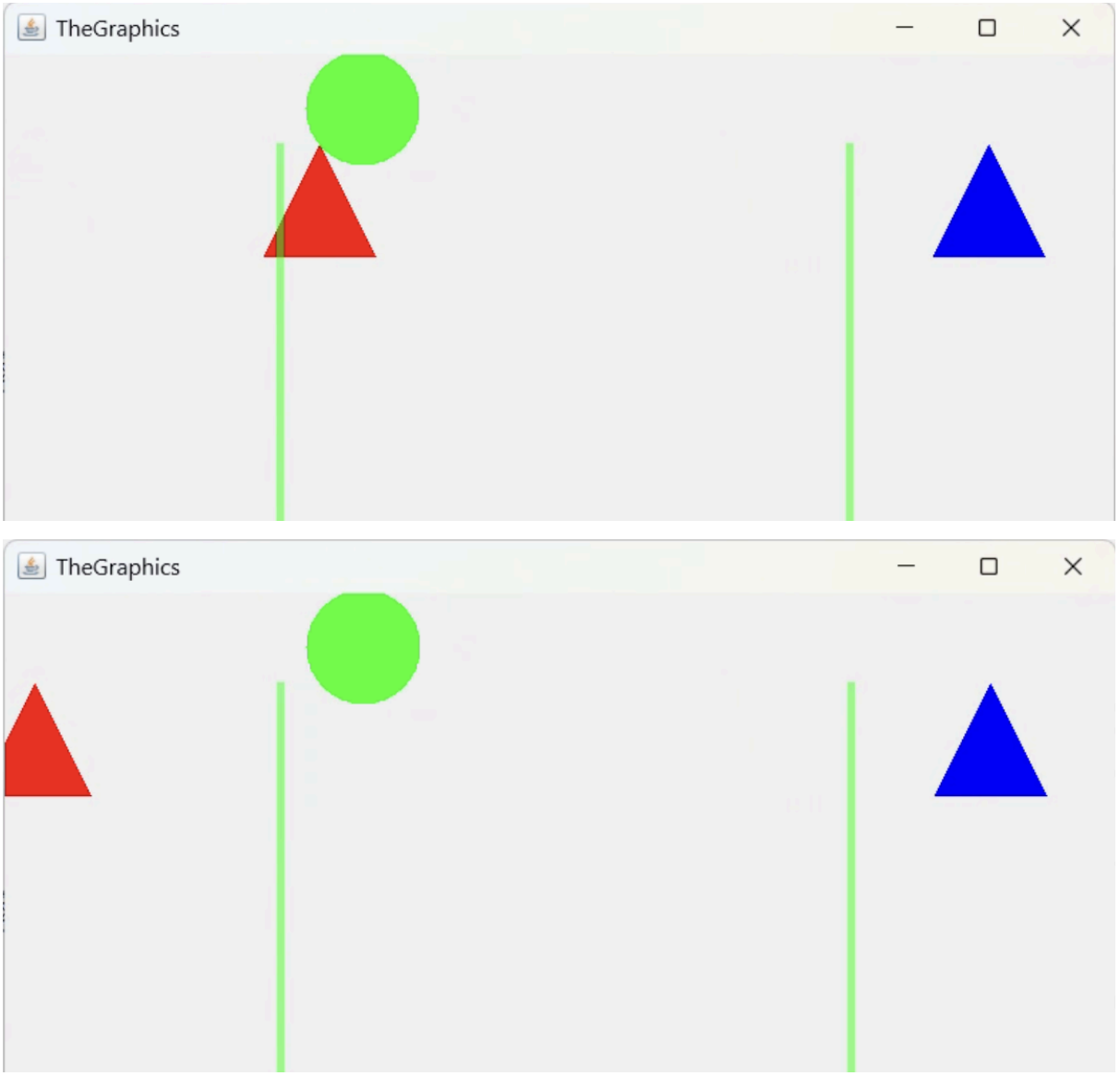
1.2. Resultado Esperado

Primero se comienza con el siguiente campo:

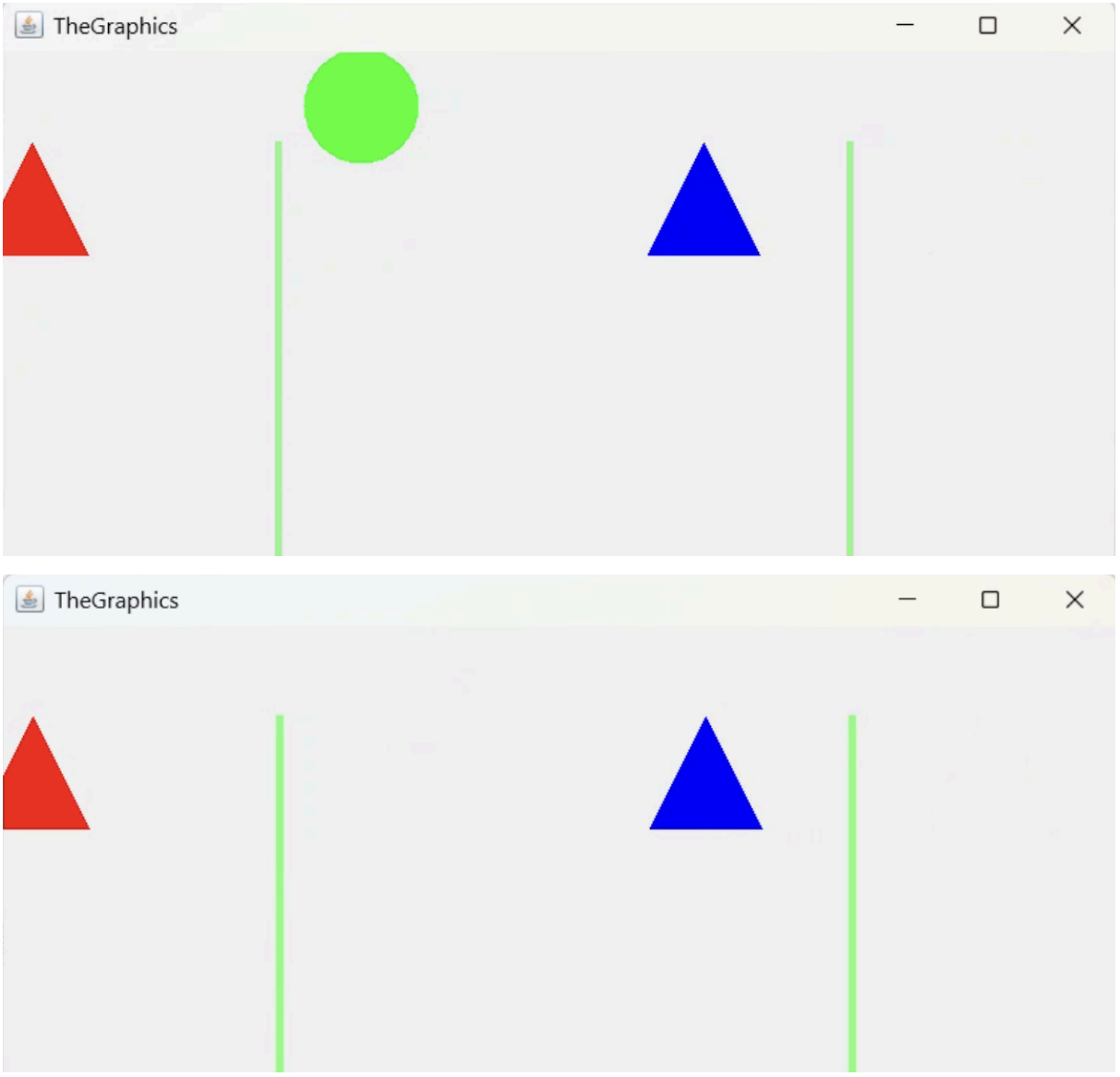


En el que el triángulo izquierdo es el productor, el triángulo derecho es el consumidor y el área central es donde estarán los paquetes.

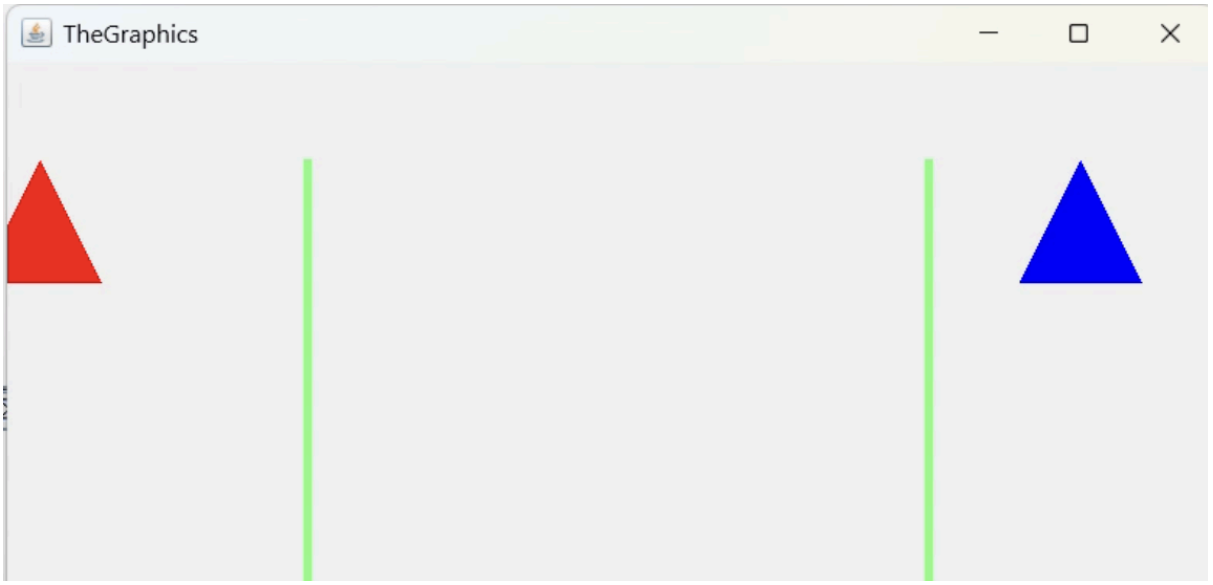
Después el productor crea un paquete:



Posteriormente ese paquete es consumido por uno de los dos consumidores:



Y finalmente se queda vacía el área central:



2. Justificación e Identificación de Tareas

El programa desarrollado consiste en una simulación del problema clásico de Productor-Consumidor con buffer limitado, implementada en lenguaje C-- y ejecutada mediante el entorno gráfico jBACI.

La aplicación representa un productor que fabrica paquetes y dos consumidores que se encargan de recogerlos y trasladarlos hasta una caja. Los paquetes se depositan en una sección crítica que dispone de una capacidad limitada de 5 piezas.

Para gestionar correctamente la producción y el consumo de paquetes, se utilizan tres semáforos que permiten controlar los espacios disponibles, las piezas pendientes de recoger y el acceso exclusivo a los recursos compartidos. De esta manera, se evita que el productor deposite más paquetes de los que admite la sección crítica o que los consumidores

intenten recoger piezas cuando no hay ninguna disponible.

La ejecución de los tres procesos se realiza de forma concurrente mediante la estructura cobegin, permitiendo que el productor y los dos consumidores trabajen simultáneamente.

El programa establece una producción total de 200 paquetes, que posteriormente son recogidos por los dos consumidores, encargándose cada uno de ellos de recoger 100 piezas.

A continuación, se identifican las principales tareas realizadas por cada uno de los procesos y los mecanismos de sincronización empleados.

2.1. Tareas realizadas por el productor

2.1.1. Inicio del proceso de producción

El productor comienza su funcionamiento mediante un bucle for que controla la fabricación de un total de 200 paquetes.

En cada iteración, muestra por pantalla un mensaje indicando el número de pieza que está produciendo, permitiendo visualizar el progreso de la producción.

2.1.2. Comprobación de los espacios disponibles

Antes de depositar un nuevo paquete, el productor ejecuta wait(huecos) para comprobar que existe espacio libre en la sección crítica.

El semáforo huecos comienza con un valor de 5, correspondiente a la capacidad máxima de la sección crítica. Cada vez que el productor deposita un paquete, el número de espacios disponibles disminuye.

Si el semáforo alcanza el valor cero, el productor queda bloqueado hasta que alguno de los consumidores retire una pieza y libere un espacio mediante signal(huecos).

2.1.3. Acceso exclusivo a la cinta transportadora

Una vez comprobada la existencia de espacio libre, el productor solicita acceso exclusivo mediante wait(mutex).

Este mecanismo impide que los consumidores modifiquen simultáneamente los recursos compartidos mientras el productor está depositando un nuevo paquete.

2.1.4. Creación y depósito de los paquetes

Con el acceso a la sección crítica garantizado, el productor crea un nuevo paquete mediante la instrucción create().

Cada paquete se representa gráficamente mediante un círculo de color verde y dimensiones de 80x80 píxeles. Además, se le asigna un identificador único, que se incrementa después de cada creación mediante la variable idPieza.

Posteriormente, el productor se desplaza 200 unidades hacia la derecha mediante moveby(), simulando el movimiento necesario para depositar el paquete.

Una vez realizado el depósito, libera el semáforo mutex mediante signal(mutex), permitiendo que otro proceso pueda acceder a la sección crítica.

Finalmente, el productor vuelve a su posición inicial y ejecuta signal(paquetes) para notificar que existe una nueva pieza disponible para los consumidores.

2.1.5. Finalización de la producción

El procedimiento anterior se repite hasta completar las 200 iteraciones del bucle.

Cuando se han producido todos los paquetes, la función finaliza su ejecución, dejando que los consumidores terminen de recoger las piezas que todavía puedan permanecer pendientes.

2.2. Tareas realizadas por los consumidores

2.2.1. Inicio del proceso de consumo

Cada consumidor ejecuta la función consumidor(), que contiene un bucle for encargado de controlar la recogida de 100 paquetes.

Como la función se ejecuta dos veces mediante cobegin, el programa dispone de dos procesos consumidores que trabajan concurrentemente.

En cada iteración, el consumidor muestra por pantalla un mensaje indicando el número de paquete que está buscando.

2.2.2. Comprobación de las piezas disponibles

Antes de recoger un paquete, el consumidor ejecuta wait(paquetes) para comprobar que existe alguna pieza disponible.

El semáforo paquetes comienza con un valor de 0, puesto que inicialmente no se ha producido ningún paquete.

Cada vez que el productor deposita una pieza, incrementa este semáforo mediante signal(paquetes).

Si no existen piezas disponibles, el consumidor queda bloqueado hasta que el productor produzca un nuevo paquete y notifique su disponibilidad.

2.2.3. Acceso exclusivo a la sección crítica

Una vez comprobada la existencia de paquetes, el consumidor ejecuta wait(mutex) para solicitar acceso exclusivo.

Este mecanismo garantiza que solamente un proceso pueda modificar los recursos compartidos en cada momento, evitando que los dos consumidores o el productor manipulen simultáneamente los paquetes.

2.2.4. Recogida y traslado de los paquetes

Una vez obtenido el acceso exclusivo, el consumidor se desplaza 200 unidades hacia la izquierda mediante

moveby(), simulando su aproximación a la sección crítica.

Posteriormente, actualiza el identificador de la pieza mediante idPieza--, obteniendo así el identificador del último paquete creado que todavía se encuentra pendiente de recogida.

A continuación, utiliza moveby() para desplazar el paquete correspondiente 700 unidades hacia la derecha, simulando su traslado hasta la caja.

Una vez recogida la pieza, el consumidor vuelve a su posición inicial mediante otro desplazamiento de 200 unidades hacia la derecha.

2.2.5. Liberación de recursos y notificación de espacios disponibles

Cuando termina de recoger el paquete, el consumidor libera el acceso exclusivo mediante signal(mutex).

Posteriormente, ejecuta signal(huecos) para incrementar el número de espacios disponibles, notificando que se ha retirado una pieza.

Finalmente, muestra por pantalla un mensaje indicando que el paquete ha sido depositado en la caja.

2.2.6. Finalización del consumo

El procedimiento anterior se repite hasta que cada consumidor ha recogido sus 100 paquetes correspondientes.

Una vez completadas las 100 iteraciones, cada proceso consumidor finaliza su ejecución.

2.3. Mecanismos de sincronización empleados

Para garantizar el correcto funcionamiento de la aplicación, se utilizan tres semáforos que coordinan las operaciones de producción y consumo.

2.3.1. Semáforo huecos

El semáforo huecos controla el número de espacios libres que existen en la sección crítica.

Su valor inicial es 5, correspondiente a la capacidad máxima del buffer.

El productor ejecuta wait(huecos) antes de depositar cada paquete, mientras que los consumidores ejecutan signal(huecos) después de retirar una pieza.

De esta manera, se impide que el productor deposite más paquetes de los que permite la capacidad disponible.

2.3.2. Semáforo paquetes

El semáforo paquetes controla el número de piezas disponibles para ser recogidas por los consumidores.

Su valor inicial es 0, ya que inicialmente no existe ningún paquete disponible.

El productor ejecuta signal(paquetes) después de depositar cada pieza, mientras que los consumidores ejecutan wait(paquetes) antes de recoger un paquete.

Así, se evita que los consumidores intenten recoger piezas cuando todavía no existe ninguna disponible.

2.3.3. Semáforo mutex

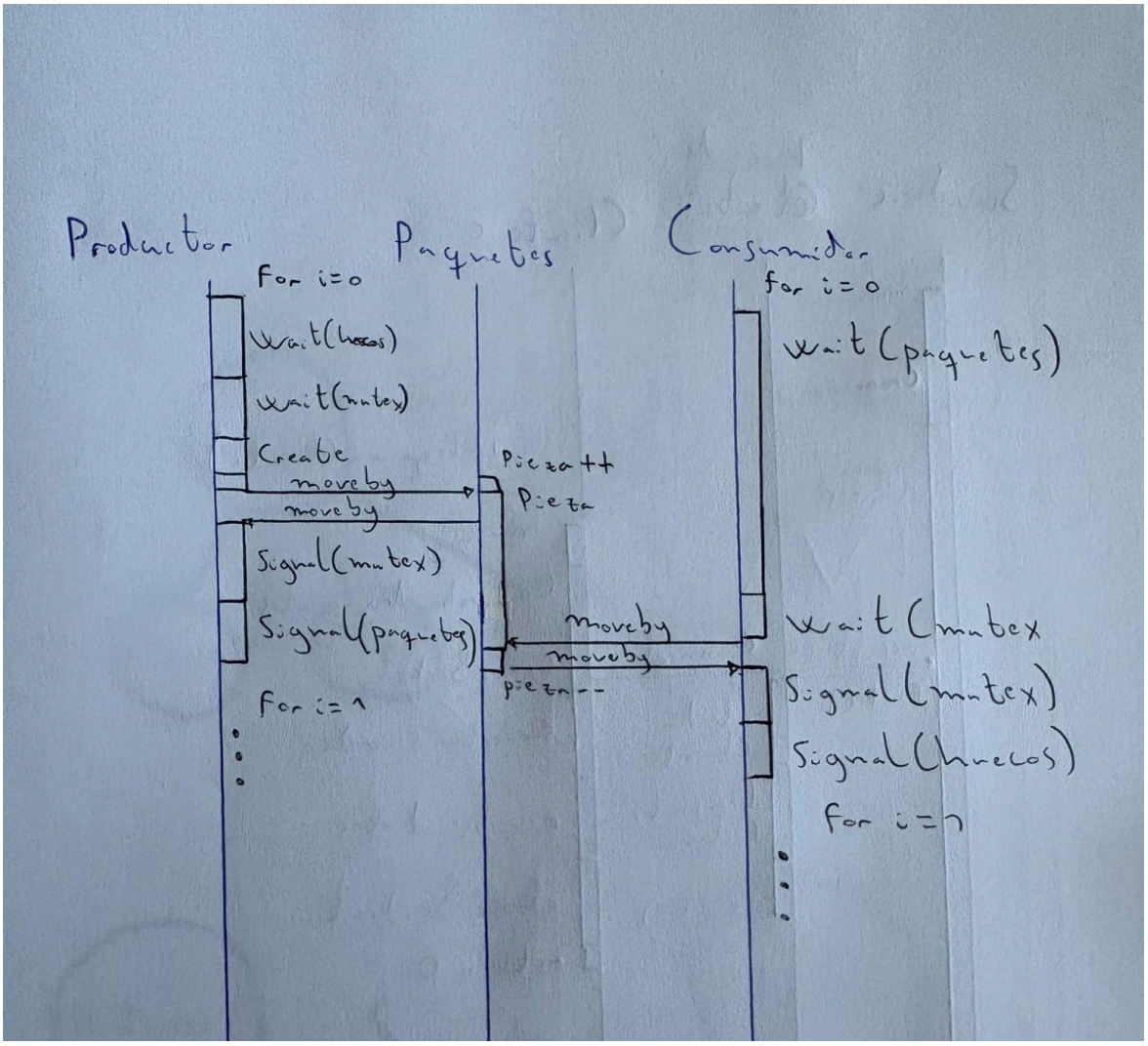
El semáforo mutex garantiza la exclusión mutua en el acceso a la sección crítica y a la variable compartida idPieza.

Su valor inicial es 1, indicando que el recurso está disponible.

Tanto el productor como los consumidores ejecutan wait(mutex) antes de modificar los recursos compartidos y signal(mutex) cuando terminan sus operaciones.

De esta manera, se evita que varios procesos modifiquen simultáneamente los identificadores de las piezas o manipulen los recursos compartidos al mismo tiempo.

3. Diagrama de Secuencia



4. Análisis detallado del Código fuente

4.1. Declaración de constantes y variables globales

Antes de definir las funciones principales, el programa declara una serie de constantes, semáforos y variables globales que permiten controlar la simulación.

En primer lugar, se incluye el archivo gdefs.cm, que contiene las definiciones necesarias para trabajar con los elementos gráficos de jBACI, como las figuras geométricas y sus colores.

A continuación, se declaran las siguientes constantes enteras:

- VARIABLE1 = 1: identificador de la figura gráfica que representa al productor.
- VARIABLE2 = 2: identificador de la figura gráfica que representa a los consumidores.
- BARRIER1 = 4: identificador de la primera barrera que delimita la zona de trabajo.
- BARRIER2 = 5: identificador de la segunda barrera que delimita la zona de trabajo.
- DELTA = 11: constante auxiliar declarada, aunque no se utiliza en el programa.
- Xbarrier1 = 200: constante auxiliar asociada a la posición horizontal de la primera barrera, aunque no se utiliza directamente.
- Xbarrier2 = 400: constante auxiliar asociada a la posición horizontal de la segunda barrera, aunque no se utiliza directamente.
- Xfinal = 600: constante auxiliar que tampoco se utiliza directamente.

Posteriormente, se declaran los tres semáforos globales:

- huecos = 5: controla los espacios libres de la sección crítica.

- paquetes = 0: controla las piezas disponibles para recoger.
- mutex = 1: garantiza la exclusión mutua en el acceso a los recursos compartidos.

Finalmente, se declara la variable entera global idPieza, inicializada a 30. Esta variable permite identificar los paquetes que se crean y posteriormente se recogen, incrementándose cada vez que el productor fabrica una pieza y decrementándose cuando un consumidor recoge una.

4.2. Función productor()

```
35  // ------------------ PRODUCTOR ------------------
36
37  void productor(){
38      int i;
39
40      // El productor fabrica un total de 100 piezas.
41      for(i = 1; i <= 200; i++){
42
43          cout << "PRODUCTOR: Produciendo pieza " << i << endl;
44
45          // Comprueba que exista espacio libre en la seccion critica.
46          wait(huecos);
47
48          // Solicita acceso exclusivo a la seccion critica.
49          wait(mutex);
50
51          // Crea un paquete circular de color verde y tamaño 80x80.
52          create(idPieza, CIRCLE, GREEN, 220, 20 + (idPieza - 29) * 15, 80, 80);
53
54          idPieza++;
55
56          // El productor avanza para depositar la pieza.
57          moveby(VARIABLE1, 200, 0);
58
59          cout << "PRODUCTOR: dejando paquete " << i << endl;
60
61          // Libera el acceso a la seccion critica.
62          signal(mutex);
63
64          // El productor vuelve a su posicion inicial.
65          moveby(VARIABLE1, -200, 0);
66
67          // Notifica que hay una nueva pieza disponible.
68          signal(paquetes);
69      }
70  }
```

Esta función representa el proceso productor de la aplicación y se encarga de fabricar los paquetes y depositarlos en la sección crítica.

No recibe ningún parámetro ni devuelve ningún valor, ya que su finalidad consiste en ejecutar directamente las operaciones de producción y sincronización.

En primer lugar, declara la variable entera i, que se utiliza como contador del bucle de producción.

A continuación, comienza un bucle for que se ejecuta desde 1 hasta 200, estableciendo así el número total de paquetes que debe fabricar el productor.

En cada iteración, se muestra por pantalla un mensaje indicando el número de pieza que se está produciendo mediante cout.

Posteriormente, se ejecuta wait(huecos), que comprueba que existe espacio disponible. Si el semáforo tiene un valor positivo, se decrementa y la ejecución continúa. En caso contrario, el productor queda bloqueado hasta que alguno de los consumidores libere un espacio.

Una vez obtenido un espacio disponible, se ejecuta wait(mutex) para solicitar acceso exclusivo a la sección crítica.

A continuación, se utiliza la instrucción:

```
51          // Crea un paquete circular de color verde y tamaño 80x80.
52          create(idPieza, CIRCLE, GREEN, 220, 20 + (idPieza - 29) * 15, 80, 80);
```

Esta instrucción crea un nuevo paquete representado mediante un círculo de color verde, con unas dimensiones de 80x80 píxeles.

El identificador del paquete corresponde al valor actual de idPieza. Además, sus coordenadas de

creación se establecen mediante los valores 220 y 20 + (idPieza - 29) * 15.

Una vez creado el paquete, se incrementa idPieza mediante idPieza++, preparando el identificador para la siguiente pieza que se produzca.

Posteriormente, el productor ejecuta moveby(VARIABLE1, 200, 0), desplazando la figura que lo representa 200 unidades hacia la derecha, simulando el movimiento de depósito del paquete.

A continuación, muestra por pantalla un mensaje indicando que está dejando el paquete correspondiente.

Cuando termina esta operación, ejecuta signal(mutex) para liberar el acceso exclusivo y permitir que otro proceso pueda utilizar la sección crítica.

Después, el productor vuelve a su posición inicial mediante moveby(VARIABLE1, -200, 0), desplazándose 200 unidades hacia la izquierda.

Finalmente, ejecuta signal(paquetes) para incrementar el número de piezas disponibles y notificar a los consumidores que existe un nuevo paquete que pueden recoger.

Este procedimiento se repite hasta completar las 200 piezas establecidas en el bucle for, momento en el que la función finaliza su ejecución.

4.3. Función robot()

```
72  // ------------------ CONSUMIDOR ------------------
73
74  void consumidor(){
75      int i;
76
77      // Cada consumidor recoge un total de 100 piezas.
78      for(i = 1; i <= 100; i++){
79
80          cout << "CONSUMIDOR: buscando paquete " << i << endl;
81
82          // Espera hasta que haya una pieza disponible.
83          wait(paquetes);
84
85          // Solicita acceso exclusivo a la seccion critica.
86          wait(mutex);
87
88          // El consumidor se desplaza hacia la seccion critica.
89          moveby(VARIABLE2, -200, 0);
90
91          cout << "CONSUMIDOR: cogiendo paquete " << i << endl;
92
93          // Actualiza el identificador de la pieza.
94          idPieza--;
95
96          // Desplaza la pieza hacia la caja.
97          moveby(idPieza, 700, 0);
98
99          // El consumidor vuelve a su posicion inicial.
100         moveby(VARIABLE2, 200, 0);
101
102         // Libera el acceso a la seccion critica.
103         signal(mutex);
104
105         // Notifica que hay un nuevo espacio disponible.
106         signal(huecos);
107
108         cout << "CONSUMIDOR: dejando paquete en caja" << endl;
109     }
110 }
```

Esta función representa el proceso consumidor de la aplicación y se encarga de recoger los paquetes depositados por el productor y trasladarlos hasta la caja.

No recibe ningún parámetro ni devuelve ningún valor.

En primer lugar, declara la variable entera i, que se utiliza como contador del número de paquetes recogidos.

A continuación, comienza un bucle for que se ejecuta desde 1 hasta 100, estableciendo que cada ejecución de esta función debe recoger un total de 100 paquetes.

En cada iteración, se muestra por pantalla un mensaje indicando el número de paquete que está buscando el consumidor.

Posteriormente, se ejecuta wait(paquetes) para comprobar que existe alguna pieza disponible.

Si el semáforo tiene un valor positivo, se decrementa y el consumidor puede continuar con su ejecución. En caso contrario, queda bloqueado hasta que el productor deposite un nuevo paquete y ejecute signal(paquetes).

Una vez confirmada la existencia de una pieza disponible, el consumidor ejecuta wait(mutex) para solicitar acceso exclusivo a la sección crítica.

A continuación, se ejecuta moveby(VARIABLE2, -200, 0), desplazando la figura que representa al consumidor 200 unidades hacia la izquierda, simulando su aproximación a la sección crítica.

Posteriormente, muestra por pantalla un mensaje indicando que está recogiendo el paquete correspondiente.

Una vez situado en la sección crítica, se ejecuta idPieza--, decrementando el identificador global de las piezas. De esta manera, se obtiene el identificador del último paquete creado que todavía no ha sido recogido.

A continuación, se utiliza la instrucción:

```
96          // Desplaza la pieza hacia la caja.
97          moveby(idPieza, 700, 0);
```

Esta operación desplaza el paquete identificado mediante idPieza 700 unidades hacia la derecha, simulando su traslado desde la sección crítica hasta la caja.

Una vez trasladada la pieza, el consumidor vuelve a su posición inicial mediante moveby(VARIABLE2, 200, 0), desplazándose 200 unidades hacia la derecha.

Cuando termina de realizar estas operaciones, ejecuta signal(mutex) para liberar el acceso exclusivo,

permitiendo que el productor u otro consumidor puedan acceder a la sección crítica.

Posteriormente, ejecuta signal(huecos) para incrementar el número de espacios libres, notificando que se ha retirado un paquete.

Finalmente, muestra por pantalla un mensaje indicando que el paquete ha sido depositado en la caja.

Este procedimiento se repite hasta que el consumidor ha recogido sus 100 paquetes correspondientes. Una vez completadas las iteraciones del bucle, la función finaliza su ejecución.

4.4. Función main()

```
112  // ------------------ PROGRAMA PRINCIPAL ------------------
113
114  void main(){
115
116      // Productor representado mediante un triangulo rojo.
117      create(VARIABLE1, TRIANGLE, RED, 30, 100, 80, 80);
118
119      // Consumidores representados mediante triangulos azules.
120      create(VARIABLE2, TRIANGLE, BLUE, 700, 100, 80, 80);
121
122      // Barreras que delimitan la zona de trabajo.
123      create(BARRIER1, RECTANGLE, GREEN, 200, 100, 5, 800);
124      create(BARRIER2, RECTANGLE, GREEN, 600, 100, 5, 800);
125
126      // Ejecucion concurrente del productor y los dos consumidores.
127      cobegin{
128          productor();
129          consumidor();
130          consumidor();
131      }
132  }
```

Es la función principal del programa y se encarga de inicializar los elementos gráficos de la simulación y poner en funcionamiento los tres procesos concurrentes.

En primer lugar, utiliza la instrucción create() para crear las figuras que representan a los distintos elementos de la aplicación.

La primera figura corresponde al productor. Se representa mediante un triángulo de color rojo, identificado por VARIABLE1, situado inicialmente en las coordenadas (30, 100) y con unas dimensiones de 80x80 píxeles.

A continuación, se crea la figura que representa a los consumidores. Se utiliza un triángulo de color azul, identificado por VARIABLE2, situado inicialmente en las coordenadas (700, 100) y con unas dimensiones de 80x80 píxeles.

Posteriormente, se crean dos barreras de color verde mediante las instrucciones create():

- BARRIER1: representa la primera barrera, situada en las coordenadas (200, 100), con unas dimensiones de 5x800 píxeles.
- BARRIER2: representa la segunda barrera, situada en las coordenadas (600, 100), también con unas dimensiones de 5x800 píxeles.

Estas barreras delimitan visualmente la zona de trabajo del productor y los consumidores.

Una vez inicializados todos los elementos gráficos, se utiliza la estructura cobegin para iniciar la ejecución concurrente de los tres procesos:

- productor(): ejecuta el proceso productor, encargado de fabricar los 200 paquetes.
- consumidor(): ejecuta el primer proceso consumidor, encargado de recoger 100 paquetes.
- consumidor(): ejecuta el segundo proceso consumidor, encargado de recoger otros 100 paquetes.

La estructura cobegin permite que las tres funciones se ejecuten concurrentemente, compartiendo las variables globales y los semáforos declarados anteriormente.

Gracias a los mecanismos de sincronización implementados, los tres procesos pueden trabajar simultáneamente sin superar la capacidad de la sección crítica ni acceder de forma concurrente a los recursos compartidos.

Finalmente, cuando los tres procesos han completado sus respectivas tareas, termina la ejecución del programa.